

Quality Assessment of Python Tests Generated by Large Language Models

Victor Alves
Federal University of Ceará (UFC)
Quixadá, Ceará, Brazil
anthonyvictor90@gmail.com

Carla Bezerra
Federal University of Ceará (UFC)
Quixadá, Ceará, Brazil
carlailane@ufc.br

Ivan Machado
Federal University of Bahia (UFBA)
Salvador, Brazil
ivanmachado@gmail.com

Larissa Rocha
University of the State of Bahia
(UNEB)
Alagoinhas, Bahia, Brazil
lari.rsoares@gmail.com

Tassio Virgínio
Federal University of Bahia (UFBA)
Salvador, Bahia, Brazil
tassio.virginio@ufba.br

Publio Silva
Federal University of Ceará (UFC)
Quixadá, Ceará, Brazil
publio.blenilio@gmail.com

Abstract

The manual generation of test scripts is a time-intensive, costly, and error-prone process, indicating the value of automated solutions. Large Language Models (LLMs) have shown great promise in this domain, leveraging their extensive knowledge to produce test code more efficiently. This study investigates the quality of Python test code generated by three LLMs: GPT-4o, Amazon Q, and LLama 3.3. We evaluate the structural reliability of test suites generated under two distinct prompt contexts: Text2Code (T2C) and Code2Code (C2C). Our analysis includes the identification of errors and test smells, with a focus on correlating these issues to inadequate design patterns. Our findings reveal that most test suites generated by the LLMs contained at least one error or test smell. Assertion errors were the most common, comprising 64% of all identified errors, while the test smell *Lack of Cohesion of Test Cases* was the most frequently detected (41%). Prompt context significantly influenced test quality; textual prompts with detailed instructions often yielded tests with fewer errors but a higher incidence of test smells. Among the evaluated LLMs, GPT-4o produced the fewest errors in both contexts (10% in C2C and 6% in T2C), whereas Amazon Q had the highest error rates (19% in C2C and 28% in T2C). For test smells, Amazon Q had fewer detections in the C2C context (9%), while LLama 3.3 performed best in the T2C context (10%). Additionally, we observed a strong relationship between specific errors, such as assertion or indentation issues, and test case cohesion smells. These findings demonstrate opportunities for improving the quality of test generation by LLMs and highlight the need for future research to explore optimized generation scenarios and better prompt engineering strategies.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Artificial Intelligence** → *Large Language Models*; • **General and reference** → Experimental studies.



This work is licensed under a Creative Commons Attribution 4.0 International License.
EASE '25, Istanbul, Türkiye
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1385-9/25/06
<https://doi.org/10.1145/3756681.3756964>

Keywords

Large Language Models, Python Test Code, Test Smells, Prompt Engineering

ACM Reference Format:

Victor Alves, Carla Bezerra, Ivan Machado, Larissa Rocha, Tassio Virgínio, and Publio Silva. 2025. Quality Assessment of Python Tests Generated by Large Language Models. In *Evaluation and Assessment in Software Engineering (EASE '25)*, June 17–20, 2025, Istanbul, Türkiye. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3756681.3756964>

1 Introduction

The development of test scripts is an essential aspect of software engineering, ensuring the reliability and correctness of software systems [15]. However, creating test scripts manually is a time-consuming, labor-intensive, and expensive process, often prone to human error [20]. Consequently, researchers have extensively explored methods for automating test generation [35, 37], leading to significant advancements in the field. Among these advancements, Artificial Intelligence (AI) tools have emerged as valuable assets in supporting automated code generation. By utilizing techniques such as search-based optimization and feedback-driven learning, these tools have enhanced their ability to produce efficient and accurate outputs [17].

Recent studies in Large Language Models (LLMs) have introduced new possibilities for test code generation [12, 13, 25, 34, 36, 44]. Models such as GPT-4, Amazon Q, and LLama 3.3 have demonstrated their ability to leverage extensive pre-trained knowledge and contextual understanding to automate the generation of test scripts. These models can simulate various test scenarios, identify potential issues, and improve testing efficiency compared to traditional approaches [45]. However, the effectiveness of LLM-generated test scripts is influenced by several factors, including the context and structure of the input prompts [12].

Prompt engineering, the process of crafting effective prompts to guide model outputs, has proven to be a critical factor in optimizing the performance of LLMs [24, 32]. Well-designed prompts can significantly improve the quality, structure, and usability of generated test code [6, 32]. Research has shown that LLMs can adapt to different types of prompts, such as textual descriptions (Text2Code) and pre-existing code snippets (Code2Code), producing varying levels of quality depending on the generation context

[1, 22]. This adaptability highlights the transformative potential of LLMs in automating software testing.

Despite their capabilities, LLM-generated test scripts often exhibit issues related to errors and test smells [36]. Like production code, test code must adhere to proper programming practices to ensure reliability and maintainability [40]. Test smells, which refer to suboptimal design or implementation choices in test code, can reduce the effectiveness of tests in detecting faults and validating software behavior [37, 38]. For instance, studies have identified significant numbers of test smells in LLM-generated tests, such as those produced by GitHub Copilot [3]. Common problems include assertion errors, poor cohesion among test cases, and inadequate error handling, highlighting the need for further investigation into LLM performance across diverse contexts.

This study builds upon the work of Alves et al. [3], extending the analysis of LLM-generated test code quality across three prominent models: GPT-4o, Amazon Q, and Llama 3.3. We compare the performance of these models in two prompt contexts, Text2Code and Code2Code, to evaluate their ability to produce high-quality Python test scripts. Our research focuses on the structural reliability of the generated tests, analyzing errors and test smells to identify patterns associated with suboptimal design. By investigating the relationship between prompt contexts, errors, and test smells, we aim to uncover opportunities for improving LLM-based test generation and provide actionable insights for researchers and practitioners.

In addition to evaluating the performance of the models, this work emphasizes the role of prompt engineering as a mechanism for enhancing the usability and reliability of automated test generation. As LLMs continue to advance, understanding their limitations and optimizing their use in test engineering are critical steps toward improving software quality and reducing development costs.

2 Background

2.1 Unit Testing & Test Smells

Unit testing aims to identify defects and verify the functionality of the smallest components of software [16]. Automated frameworks like JUnit¹ have made this approach widely adopted, allowing for the frequent and automatic execution of unit test suites [10]. This practice is crucial to avoid programming mistakes and detect problems early in the development process [18, 29].

In the realm of unit testing, specific issues known as test smells can significantly undermine the quality and effectiveness of test cases. These smells typically arise from suboptimal design decisions made during the creation of test cases, which can greatly reduce test effectiveness [28, 39]. Test smells often emerge when test code is first added to a repository and tend to persist, negatively impacting the software's maintainability and directly affecting its quality [19, 33, 37]. The presence of test smells in test code can directly compromise internal quality attributes such as code cohesion, complexity and maintainability [11]. The literature presents various approaches to detecting test smells, ranging from natural language-based techniques for removing test smells [4] to automated tools that analyze tests and provide quantitative data on the identified test smells [5, 14, 27, 40].

¹<https://junit.org/junit5/>

2.2 Test Generation by LLMs

Automated test generation facilitates the production and execution of numerous inputs that thoroughly test software units [42]. Traditional approaches to test generation have included model-driven, requirement-based, static analysis, and research-driven techniques [23, 26].

LLMs have assumed a significant role in test generation driven by Natural Language Processing (NLP) [45]. The performance of the code generated shows little variation between the different language models, regardless of their size or the data set used for training [9]. By training in human language input, LLMs such as GPT-4o² and Llama 3.3³ can generate code snippets, documentation, and even repair bugs [25]. This functionality is exemplified in applications like GitHub Copilot and ChatGPT, which have been extensively studied for their capability in test code generation [12, 34]. In addition, some frameworks and techniques have recently been studied to facilitate test generation by LLMs [2, 8, 41].

Despite these advancements, significant challenges remain in evaluating the quality of code generated by these models [21]. While they hold promise in automating repetitive coding tasks and enhancing productivity, understanding the reliability, correctness, and maintainability of the generated test code is critical. Addressing this gap is essential to unlocking the full potential of LLMs in automated software testing and ensuring their outputs align with best practices and quality standards.

2.3 Prompt Engineering

The process of prompt engineering involves strategically designing task-specific instructions, guiding the output of the model without changing the parameters [32]. Studies on response generation in prompting contexts [21, 30] highlight that the effectiveness of prompts can vary significantly based on the specific context and objectives. These studies emphasize that the design of instructions is pivotal in producing accurate and reliable outputs.

Additionally, recent research on process automation with LLMs [1, 22, 31] has explored various types of prompt contexts that influence the outputs generated by LLMs. These studies have characterized different prompt strategies and their applications, shedding light on how carefully designed prompts can optimize the capabilities of LLMs to meet task-specific requirements. Among the commonly discussed prompt contexts are:

- **Code to Code (C2C):** context in which the prompt is a code snippet and the output is also a code snippet. Used in production, maintenance, and code translation tasks.
- **Text to Code (T2C):** context in which the prompt is a natural language text and the output is a code snippet. Used in code generation and refactoring tasks.
- **Code to Text (C2T):** context in which the prompt is a code snippet and the output is natural language text. Used in code summarization and documentation tasks.

3 Related Work

El Haji et al. [12] conducted an experiment with the Codex [7] version of GitHub Copilot, investigating the usability of different

²<https://openai.com/index/hello-gpt-4o/>

³<https://www.llama.com/>

test cases generated using command prompts. They found that a comment combining instructive natural language with an example of code usage resulted in more usable test generations. Similarly, Yu et al. [45] analyzed ChatGPT's ability to generate mobile test scripts. Their findings indicate that ChatGPT can generate useful tests when provided with sufficient context and information about the project architecture. Alves et al. [3] investigated the presence of test smells in OSS project tests generated by LLMs in a single generation and prompt context. Although the authors of the Alves et al. [3] study carried out a code quality analysis, this analysis was limited to one LLM (Github Copilot with GPT-4) and one prompt. In this work, we used two prompt contexts and correlated the occurrence of errors in the code with test smells.

Several studies have already been conducted to assess the quality of the production code. Yetistiren et al. [43] examined the quality of the code produced by the GitHub Copilot. They found that while Copilot can generate syntactically valid code, many of the generated solutions still exhibit issues related to efficiency and design. In complementary research, Yetistiren et al. [44] investigated the presence of code smells in codes generated by Amazon's CodeWhisperer, Copilot and ChatGPT. They discovered that certain code smells frequently reappear in the generated code and that the LLMs themselves can solve these maintenance issues. This recurring pattern of problems affecting code maintainability was also noted by Hansson and Ellréus [17], who highlighted that ChatGPT and GitHub Copilot have been more effective in producing quality code. Therefore, although numerous evaluations of the quality of code generated by LLMs exist, these analyses have primarily focused on production code and have not specifically examined test code. From these studies, we extracted code quality analysis techniques and adapted them for test code.

4 Study Design

4.1 Goals and Research Questions

This study aims to evaluate the quality of the test suites generated by LLMs in Python. The evaluation focuses on both the syntactic and semantic quality of the generated code, as well as the structure of the test cases, with particular attention to identifying potential design irregularities. Initially, we assessed the validity of the tests by identifying errors. Subsequently, we measured the structural quality using test smell metrics. These evaluations were conducted across multiple LLMs and in two distinct prompt contexts: C2C and T2C. To guide our analysis, we formulated four key research questions:

- RQ₁:** *What are the types and frequencies of errors reported in the test suites generated by LLMs during execution?* This question seeks to identify the errors that occur during the execution of the test suites generated by the models in Python. Errors can include syntax problems, runtime exceptions or failures in the test logic.
- RQ₂:** *What is the distribution and what are the specific types of test smells identified in the test suites generated by LLMs?* The second question investigates the presence of test smells in the code. The analysis includes identifying the most common types and their distribution among the different test suites.

RQ₃: *How does the type of prompt influence the occurrence of errors and test smells in test suites generated by LLMs?* In this question, the focus is on the impact of the type of prompt used (T2C or C2C) on the quality of the tests generated. The question assesses whether different input styles result in significant variations in the frequency of errors and test smells.

RQ₄: *Is there a correlation between the presence of errors and the detection of test smells in the same test suite generated by LLMs?* This question explores the correlation between the presence of execution errors and test smells in the same test suite. The aim is to see if there is a pattern in which structurally problematic tests (with test smells) are more likely to present errors.

4.2 Study Settings

We divided this study into four phases. First, we selected the dataset to be utilized. Next, we formulated the prompts. In the third phase, we generated the test suites using three LLMs. Finally, we identified errors, test smells, and co-occurrences to address the research questions. All the steps are outlined and mapped in Figure 1.

4.2.1 Dataset Selection. Initially, the codes and descriptions contained in the HumanEval⁴ dataset were extracted and stored separately to facilitate the subsequent stages of the study. HumanEval, developed by OpenAI, was designed to train and evaluate LLMs' ability to generate functional code from natural language instructions and code examples. This dataset is made up of a collection of Python programming problems, each containing the final code, a textual description, input examples, and expected output.

4.2.2 Preparing prompts. After extracting the codes from the dataset and their respective textual descriptions, two different types of prompts were created in this study: one focused on code and the other on text. The code prompt was composed predominantly of code extracted from HumanEval. Only one natural language instruction was added to guide test generation, ensuring the main focus remained on code analysis. In the text prompt, in addition to the textual description of the code provided by HumanEval, the prompt included a specific instruction to guide the generation of the tests; in this second context, the LLMs did not have access to the code, only to the text in natural language.

Listings 1 and 2 show examples of the prompts generated from the codes (C2C) and descriptions (T2C) contained in HumanEval. Listing 1 shows an example of a textual prompt, including the name of the production code function, its description, and a specific instruction for generating the unit test. Listing 2 displays a prompt containing only the instruction and the production code. It is worth noting that the code provided in this second prompt corresponds to the description used in the previous textual prompt, ensuring equivalence between the two input contexts.

```
from typing import List
def separate_paren_groups(paren_string: str) -> List[str]:
```

⁴<https://github.com/openai/human-eval>

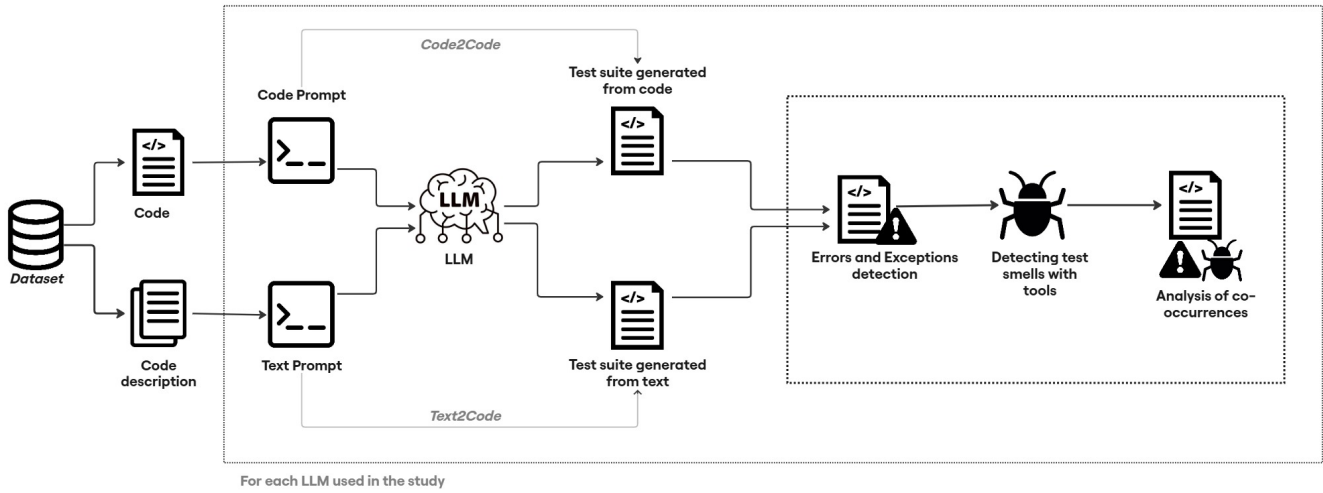


Figure 1: Methodological procedures.

```
The input to this function is a string containing several groups
of nested parentheses. Its purpose is to separate these
groups into separate strings and return a list of them. The
separated groups are balanced (each open brace is closed
correctly) and are not nested in each other. >>>
separate_paren_groups('(') ( ) ( ( ) ( ( ) ( ) )' ['()', '(() )',
'(()())']
```

```
Write a unit test for this function according to the
specifications
```

Listing 1: Example of text prompt used in the study.

```
Write a unit test for this function:

from typing import List

def separate_paren_groups(paren_string: str) -> List[str]:
    result = []
    current_string = []
    current_depth = 0

    for c in paren_string:
        if c == '(':
            current_depth += 1
            current_string.append(c)
        elif c == ')':
            current_depth -= 1
            current_string.append(c)
            if current_depth == 0:
                result.append(''.join(current_string))
                current_string.clear()

    return result
```

Listing 2: Example of code prompt used in the study.

4.2.3 Test Code Generation. The prompts prepared in the previous step were submitted to the LLMs so that they could generate automated test suites based on the information provided. Each item in the dataset contained a block of code accompanied by its description in natural language. Both were submitted separately to the LLMs so that, in the end, each code resulted in a pair of test suites: one generated based on the code (Code2Code context) and the other based on the textual description (Text2Code context). At the end of generation, these codes from the two prompt contexts were stored to be analyzed separately. For this study, three LLMs

were used: LLaMA⁵, version 3.3 70B; Amazon Q Developer⁶; and GitHub Copilot⁷, version GPT-4o.

4.2.4 Analysis of Code Errors. Once the test suites had been generated, they were run to check for interpretation errors. During this process, all errors reported by the Python interpreter were identified and mapped. To categorize the errors detected, the official Python documentation on Errors⁸ and Exceptions⁹ was used as a reference, allowing for a detailed characterization of each error present in the generated suites. In this step, the distribution of error detection was counted and the error types were classified in relation to each prompt context, considering the results obtained for each of the three LLMs analyzed. To count the errors and exceptions, we initially added up all the detections identified in the overall set. Subsequently, we calculated the specific total for each type of error identified in each LLM, allowing for a segmented evaluation of the results, which is described in Equation 1.

$$T_{\text{errors}} = \sum_{i=1}^m \text{Error}_i \quad (1)$$

In total, we found 151 detections of errors in the test suites. After counting all the errors, we calculated the percentage distribution of errors by type, as described in Equation 2. The aim of this calculation is to determine the distribution of errors by type based on the total errors detected in each LLM (calculated in Equation 1) in relation to the overall number of errors identified across all LLMs.

$$\text{Distrib}_{\text{errors}} = \frac{T_{\text{errors}}}{\sum_{\text{LLMs}} T_{\text{errors}}} \quad (2)$$

The errors detected in this study are categorized in Table 1, which presents the error types along with their descriptions based on the Python language documentation. We identified eight main

⁵<https://www.llama.com/>

⁶<https://aws.amazon.com/pt/q/developer/>

⁷<https://github.com/features/copilot>

⁸<https://docs.python.org/3/tutorial/errors.html>

⁹<https://docs.python.org/3/library/exceptions.html>

types of errors in the generated tests: *AssertionError*, *SyntaxError*, *IndentationError*, *KeyError*, *AttributeError*, *NameError*, *TypeError*, and *OverflowError*.

Table 1: Errors and Exceptions detected

Error	Description
<i>AssertionError</i>	Raised when an assert statement fails.
<i>SyntaxError</i>	Raised when the code does not follow the syntax rules of the Python language
<i>IndentationError</i>	Raised when the alignment of the lines does not follow the blocks established
<i>KeyError</i>	Raised when a mapping (dictionary) key is not found in the set of existing keys.
<i>AttributeError</i>	Raised when an attribute reference or assignment fails.
<i>NameError</i>	Raised when a local or global name is not found. This applies only to unqualified names.
<i>TypeError</i>	Raised when an operation or function is applied to an object of inappropriate type.
<i>OverflowError</i>	Raised when the result of an arithmetic operation is too large to be represented.

4.2.5 Test Smells Detection. Once the test suites had been generated and the errors identified, the detection of test smells in the code was carried out. For this step, we previously selected two specific tools to identify inconsistencies and quality violations in unit tests: Pynose [40] and TEMPY [14]. Each adopts its own techniques and approaches to detect and categorize test smells. To count the test smells, we first summed up all the detections. Then, for each LLM, we apply the calculation of the individual sum of detections by type (see Equation 3), allowing for a detailed and segmented analysis of the results.

$$T_{\text{smells}} = \sum_{i=1}^n \text{Smell}_i \quad (3)$$

In total, we found 512 detections of smells in the test suites. After counting all the smells, we calculated the percentage distribution of smells by type (see Equation 4). The aim of this equation is to determine the distribution of test smells by type based on the total smells detected in each LLM (see 3) in relation to the overall number of test smells identified across all LLMs.

$$\text{Distrib}_{\text{smells}} = \frac{T_{\text{smells}}}{\sum_{\text{LLMs}} T_{\text{smells}}} \quad (4)$$

The test smells detected in this study are categorized in Table 2, which shows the type of test smell detected, the tool that detected it and a brief description. Some test smells were detected by both tools, but they were only counted for the calculation, ruling out the possibility of duplicates.

Table 2: Test smells detected by tools

Test smell	Tools	Description
Assertion Roulette	Pynose	Several asserts without any explanation or message [40].
Lack of Cohesion of Test Cases	Pynose	Test cases that are grouped together in a test suite, but are not cohesive. Class cohesion is a metric that indicates how well the various parts and responsibilities of a class are joined together [40].
Suboptimal Assertion	Pynose	Presence of one of the suboptimal asserts [40]
Programming Paradigms Blend	TEMPY	Mixing paradigms in the same test file [14].
Non-Functional Statement	TEMPY	Empty scope in a test method [14]
Magic Number Test	Pynose	Existence of literal numeric values in a test [40].
Unknow Test	TEMPY	Tests without assertions [14].
Conditional Test Logic	Pynose, TEMPY	Tests with control statements (if, for, while...) [40].
Duplicate Assert	Pynose	Duplicate assertions in the same test [40].

4.2.6 Co-occurrence analysis. The final stage of the study consisted of analyzing the correlation between the errors detected and the test smells identified by the tools. To do this, only codes that simultaneously presented errors and test smells in the same test suite were considered. The number of test smells detected in each test suite with an error was counted, and a relationship was established between the types of test smells identified and the types of errors reported by the Python interpreter. With this approach, it was possible to assess the co-occurrence of errors and test smells, analyzing both the frequency and distribution of each type within the affected test suites. To count the co-occurrences, we consider the sum (see Equation 6):

$$k = \sum_{\text{LLMs}} T_{\text{smells}} \quad (5)$$

$$Q_{\text{tdco-occurrence}}(S, E) = \sum_{i=1}^k \mathbb{1}(S \cap E) \quad (6)$$

Where k is the total number of cases evaluated (total number of smells detected), and $\mathbb{1}(S \cap E)$ is an indicator function that returns 1 if S (smell) and E (error) co-occur in the same case, and 0 otherwise. Finally, a total of 512 cases were analyzed, resulting in 265 co-occurrences.

5 Results

5.1 RQ1: Types and Frequency of Errors

To analyze the errors, all the test suites generated were run together with their respective production codes, previously extracted from HumanEval. Tests that ran successfully, were classified as valid. Those that failed or resulted in exceptions raised by the Python interpreter were recorded as error cases. In total, 151 errors were identified in the test suites analyzed. It is important to note that some suites had multiple errors, and each occurrence was counted separately for analysis purposes, reflecting the diversity and frequency of the faults detected.

Table 3 shows the distribution of the types of errors detected in the test suites for the C2C and T2C contexts, analyzed in each of the three LLMs. Overall, assertion errors (*AssertionError*) were

Table 3: Distribution of errors in test suites generated by LLMs

LLM	Prompt Context	Assertion Error	Indentation Error	Syntax Error	Key Error	Attribute Error	Name Error	Type Error	Overflow Error	Zero Division Error	Index Error	Value Error	TOTAL
GPT-4o	C2C	0,112	0,000	0,286	0,143	0,000	0,000	0,000	0,333	0,000	0,000	1,000	0,106
GPT-4o	T2C	0,082	0,000	0,000	0,000	0,000	0,000	0,000	0,333	0,500	0,000	0,000	0,066
Amazon Q	C2C	0,194	0,235	0,143	0,143	0,000	0,600	0,000	0,333	0,000	0,000	0,000	0,192
Amazon Q	T2C	0,184	0,765	0,286	0,286	0,667	0,400	0,500	0,000	0,000	0,000	0,000	0,285
LLama	C2C	0,276	0,000	0,000	0,143	0,167	0,000	0,500	0,000	0,500	1,000	0,000	0,219
LLama	T2C	0,153	0,000	0,286	0,286	0,167	0,000	0,000	0,000	0,000	0,000	0,000	0,132
Distribution %		0,649	0,113	0,046	0,046	0,040	0,033	0,026	0,020	0,013	0,007	0,007	1,000

the most prevalent, accounting for 64.9% of the total. This result shows that many tests were generated with incorrect expectations regarding the behavior of the production code. The second most frequent type of error was *IndentationError*, which accounted for 11.3% of detections, indicating that several parts of the generated tests did not follow the indentation standards required by Python, resulting in execution failures. *SyntaxError* and *KeyError* errors had the same detection rate (4.6%), suggesting that some suites had syntax errors or cases of “hallucinations”, where LLMs tried to access non-existent variables or functions. Other errors were also detected to a lesser extent: *AttributeError* (4%), *NameError* (3.3%), *TypeError* (2.6%), *OverflowError* (2%), *ZeroDivisionError* (1.3%), and, less frequently, *IndexError* and *ValueError* (both 0.7%).

When analyzing each LLM studied individually (Table 3), it can be seen that GPT-4o was the model with the lowest incidence of errors, totaling 10.6% and 6.6% of detections in the C2C and T2C contexts, respectively. In addition, GPT-4o had the lowest diversity of errors by type, with most occurrences concentrated in *AssertionError* and *SyntaxError*. On the other hand, Amazon Q Developer recorded a high error rate in the test suites, corresponding to 19.2% of detections in the C2C context and 28.5% in T2C. This model was also the only one responsible for *IndentationError* cases, due to many generated tests not being completed correctly. In several cases, test functions were declared with inadequate indentation, compromising their execution. LLaMA, on the other hand, had an intermediate performance, with 21.9% of errors in the C2C context and 13.2% in T2C. These results place LLaMA between the two extremes, showing a moderate distribution of errors compared to the other models.

Answer to RQ₁: We identified a total of 151 occurrences of errors in the test suites generated by the LLMs. Among these errors, the most frequent were *AssertionError* (64.9%), *IndentationError* (11.3%) and *SyntaxError* (4.6%). When analyzing the results by LLM, GPT-4o had the fewest errors, while Amazon Q Developer had the most. Notably, some types of errors, such as *IndentationError* and *NameError*, were exclusive to the tests generated by Amazon Q Developer. LLaMA, on the other hand, although it had an intermediate occurrence of errors overall, was responsible for the highest number of *AssertionError* errors. This suggests that certain models may be more reliable than others in generating valid tests, but also that specific errors, such as assertion errors, may be more prevalent in some cases, affecting the accuracy and reliability of the tests generated, since errors of this type were detected in the tests of all three LLMs.

5.2 RQ₂: Types and Distribution of Test Smells

Each test suite was analyzed using specialized tools for detecting test smells. All detections made by the two tools were counted, considering each smell identified individually, even when multiple smells were found in the same test suite. By consolidating the results from both tools, 512 detections of test smells were recorded, which served as the basis for the analysis in this study.

The results of the distribution of test smells are shown in Table 4. The analysis revealed that the most frequently identified smell was *Lack of Cohesion of Test Cases*, responsible for 41.2% of all detections. This indicates that a significant number of test suites contained cases that lacked cohesion between them, compromising the structural quality of the tests. Next, the *Assertion Roulette* smell was detected in 24.2% of cases, which suggests that many tests had multiple assertions without clear documentation or explanations, making them difficult to understand and maintain. *Programming Paradigms Blend* was the third most frequent smell, with 15.8% of detections, indicating that some tests used fields initialized outside the test class (in global scope), which is considered an inappropriate practice. Another relevant smell was the *Suboptimal Assertion*, documented by Wang et al. [40] in the Pynose tool, which obtained recurrent detections. This smell reflects assertions that were not specific, robust or relevant enough to validate the expected behavior of the production code. In addition, Table 4 highlights other less prevalent test smells: *Non-Functional Statement* and *Unknown Test* appeared in 3.7% of detections, *Conditional Test Logic* in 2.7%, *Duplicated Assert* in 1.4%, and *Exception Handling* was identified only once. These results reinforce the diversity and complexity of the design problems found in the test suites generated by LLMs.

In the analysis by LLM, Amazon Q Developer showed the greatest recurrence and diversity of test smells, standing out as the model with the greatest quantity and distribution of detections. Notably, some smells were exclusive to the code generated by this model, such as *Unknown Test*, *Non-Functional Statement*, *Duplicated Assert* and *Exception Handling*, which shows specific problems in the quality of the tests produced. On the other hand, LLaMA showed the best performance in terms of structural quality, with the lowest number and variety of smells detected. Only three types were identified in its test suites: *Lack of Cohesion of Test Cases*, *Assertion Roulette* and *Conditional Test Logic*, demonstrating greater consistency in generating tests with fewer quality violations. GPT-4o fell between these two extremes, with a moderate volume of detections and a low diversity of test smells. Only four types were identified, similar to LLaMA. However, there was a significant concentration of the

Table 4: Distribution of test smells in test suites generated by LLMs

LLM	Prompt Context	Lack of Cohesion of Test Cases	Assertion Roulette	Programming Paradigms Blend	Suboptimal Assertion	Non-Functional Statement	Conditional Test Logic	Unknown Test	Duplicated Assert	Exception Handling	TOTAL
GPT-4o	C2C	0,171	0,185	0,012	0,056	0,000	0,000	0,000	0,000	0,000	0,121
GPT-4o	T2C	0,180	0,137	0,432	0,917	0,000	0,000	0,000	0,000	0,000	0,240
Amazon Q	C2C	0,104	0,097	0,099	0,000	0,000	0,000	0,316	0,143	1,000	0,098
Amazon Q	T2C	0,180	0,315	0,444	0,028	1,000	0,143	0,684	0,857	0,000	0,301
LLama	C2C	0,185	0,202	0,012	0,000	0,000	0,357	0,000	0,000	0,000	0,137
LLama	T2C	0,180	0,065	0,000	0,000	0,000	0,500	0,000	0,000	0,000	0,104
Distribution %		0,412	0,242	0,158	0,070	0,037	0,027	0,037	0,014	0,002	1,000

Suboptimal Assertion smell, which had a high frequency of detections in one of the contexts analyzed. This pattern suggests that although GPT-4o produces less variety of smells, it may be prone to specific issues related to the quality of the assertions generated.

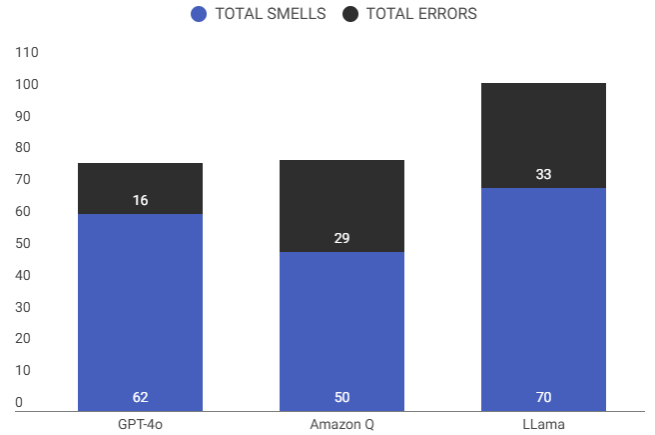
Answer to RQ₂: We identified a total of 512 detections of test smells in the test suites generated by the LLMs. *Lack of Cohesion of Test Cases* was the most frequent smell (41.2%), followed by *Assertion Roulette* (24.2%) and *Programming Paradigms Blend* (15.8%). Other less prevalent smells included *Non-Functional Statement*, *Unknown Test*, *Conditional Test Logic*, *Duplicated Assert*, and *Exception Handling*. Among the LLMs, Amazon Q Developer exhibited the highest occurrence and diversity of test smells, with several issues being unique to its generated code, such as *Unknown Test* and *Non-Functional Statement*. In contrast, LLama had the best structural quality, with only three types of smells detected, suggesting more cohesive and well-structured tests. GPT-4o showed moderate performance, with four types of smells identified but a notable concentration in *Suboptimal Assertion*, indicating frequent issues with assertion specificity and robustness.

5.3 RQ₃: Prompt Influence

Based on the analysis of Tables 3 and 4, it was possible to observe the impact of the different prompt types on the occurrence of errors and test smells in the test suites generated by each LLM. Based on this data, we conducted a detailed analysis, separating the total number of detections for each prompt context, allowing for a more in-depth understanding of the influence of context on the quality of the generations.

5.3.1 Analyzing context Code2Code. When analyzing the data from the test suites generated in the C2C context (Figure 2), we observed a moderate variation in the detection of test smells. The Amazon Q model had 50 detections, while LLama led with 70, and GPT-4o was in an intermediate position with 62 detections. As for the occurrence of errors, GPT-4o stood out for having the lowest number in this context, while the LLama (33) and Amazon Q (29) models had a similar incidence. These results indicate that, despite having generated the tests based on the production code provided in the prompt, the Amazon Q and LLama models showed a high occurrence of errors in the tests generated, which shows weaknesses in the quality of their generations in this scenario.

5.3.2 Analyzing context Text2Code. In the T2C context in Figure 3, the incidence of test smells in GPT-4o doubled compared to the

**Figure 2: Quantity of errors and test smells in the C2C context**

C2C context, reaching 123 detections, although the occurrence of errors was lower. Amazon Q had an even sharper increase, with the number of smells tripling (154 detections) and the incidence of errors rising to 43. On the other hand, LLama showed a significant improvement compared to the C2C context, with a reduction in both the number of test smells and the number of errors. These results indicate that LLama has shown greater ability to generate quality tests from textual descriptions, outperforming Amazon Q. In addition, Amazon Q faced critical difficulties in interpreting text to generate code, with some of the problems related to errors in the code produced. In the case of GPT-4o, the main challenge in the T2C context was the high incidence of test smells in the structure of the codes generated. However, the number of errors remained low, indicating a functional but poorly optimized structure.

Answer to RQ₃: The results of this analysis show the individuality of each LLM in relation to the two prompt contexts analyzed. It was not possible to identify a consistent pattern between the three models in terms of the quality of the test suites generated, since each LLM showed different behaviors.

GPT-4o stood out as the most stable model between the two contexts, with a low incidence of errors in both prompts. However, in the T2C context, there was a significant increase in the number of test smells, which suggests that the model structures the tests better by having direct access to the production code in the prompt. Amazon Q faced major difficulties in the T2C context, with

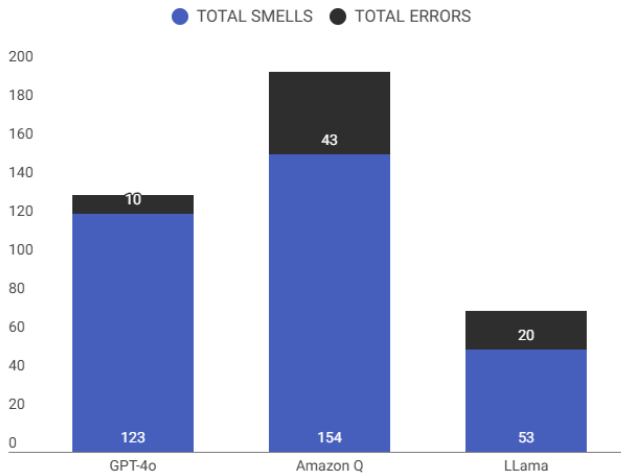


Figure 3: Quantity of errors and test smells in the T2C context

a high incidence of errors and test smells. This indicates that the model is more reliable for generating tests when it can visualize the code directly, as in the C2C context. On the other hand, LLaMa performed better when generating tests in the T2C context, with a considerable reduction in errors and test smells compared to the C2C context. This behavior indicates that the model is better able to interpret textual descriptions and generate quality tests from them.

These observations reinforce the importance of understanding the particularities of each LLM to optimize the use of the different prompt contexts, depending on the specific needs of quality and reliability in test generation.

5.4 RQ4: Correlation between Errors and Test Smells

To correlate the occurrence of errors with test smells, we accounted for all test smell detections and identified those directly associated with errors. This analysis revealed that out of 512 test smell detections, 265 were found in test suites that exhibited some type of error. Figure 4 visually demonstrates the relationships between the detected test smells and their associated errors. These results represent the aggregated detections across all three LLMs analyzed in this study.

Lack of Cohesion of Test Cases was the most frequent test smell, with 131 occurrences, and is strongly associated with *AssertionError*, which accounted for 138 occurrences. This relationship indicates that test suites with low cohesion often fail to validate the expected behavior of production code. In addition, this test smell also had connections with errors such as *IndentationError* and *SyntaxError*, showing structural and formatting problems in tests with non-cohesive cases. *Assertion Roulette*, identified 40 times, is mostly linked to *AssertionError*, highlighting that tests with multiple assertions without clear explanations considerably increased the likelihood of incorrect expectations. On the other hand, the *Programming Paradigms Blend*, with 50 detections, showed a wide dispersion of connections, including errors such as *IndentationError*

and *TypeError*. This suggests that mixing paradigms in tests can damage their structure and logical execution.

Unknown Test, with 17 occurrences, stood out for its strong association with *IndentationError*. This connection indicates that many tests classified as unknown had serious formatting problems, related to the fact that Amazon Q Developer did not structure the code properly (as discussed in RQ3). Other test smells, such as *Non-Functional Statement* (14 occurrences), have multiple connections with errors such as *KeyError* and *NameError*. These results indicate that poorly defined or non-functional tests often use non-existent or invalid elements, causing critical failures. In addition, *Duplicate Assert* and *Conditional Test Logic*, although less frequent, are associated with *AssertionError*, highlighting that problems with conditional logic or duplicate assertions directly affect the effectiveness of tests.

The less recurrent errors, such as *OverflowError*, *ZeroDivisionError*, *IndexError* and *ValueError*, appear less frequently, but maintain connections with various test smells. This suggests that specific failures may be caused by a combination of structural and logical problems. Finally, it is important to note that all the test smells detected are directly or indirectly linked to *AssertionError*, reinforcing that incorrect validation is a critical problem in the test suites generated by LLMs.

Answer to RQ4: The analysis revealed that 265 of the 512 detections of test smells occurred in tests with errors, showing a strong relationship between structural problems and test failures generated by LLMs. *Lack of Cohesion of Test Cases* was the most frequent test smell. It showed a strong association with *AssertionError*, indicating that tests with low cohesion often fail to validate the code. In addition, errors such as *IndentationError* and *SyntaxError* were common in non-cohesive tests, indicating structural flaws. *Assertion Roulette* also increased the incidence of *AssertionError*, showing that multiple assertions without context lead to incorrect expectations.

The mixture of paradigms in the tests generated a variety of errors, affecting both the structure and logic of the tests. Poorly defined tests or problematic conditional logic also contributed to critical failures. Overall, the predominance of *AssertionError* in almost all test smells reinforces that inadequate validation is a central problem in tests generated by LLMs, compromising their reliability.

6 Discussion and Implications

The results of this study have direct implications for the software testing industry, especially for teams adopting agile methodologies and practices such as Test-Driven Development (TDD). Analyzing the quality of tests generated by LLMs in Code2Code (C2C) and Text2Code (T2C) contexts reveals specific challenges for different developer profiles and test-writing approaches.

In the scenario of developers writing tests directly from code (C2C), as in traditional TDD approaches, automatic test generation can be used to speed up the development cycle. However, our results indicate that LLMs still have limitations in producing cohesive and well-structured tests, with a high incidence of test smells such as *Lack of Cohesion of Test Cases* and *Assertion Roulette*. Thus, manual review is still needed to ensure the quality of the tests generated, as

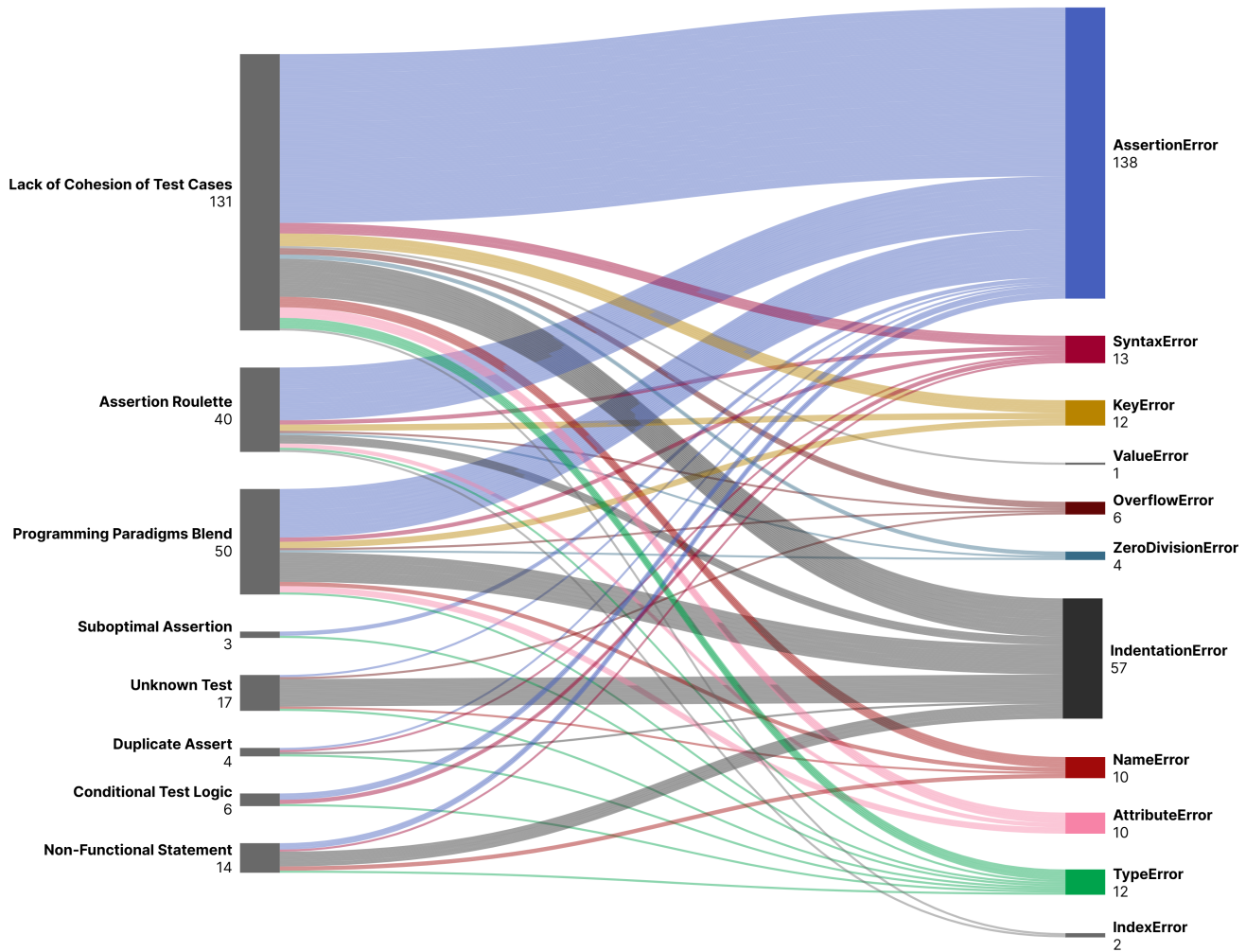


Figure 4: Cooccurrences of Errors and Test Smells by type

errors such as *AssertionError* and *IndentationError* were recurrent, requiring refinement before adoption in a real environment.

On the other hand, developers who write tests based on textual descriptions (T2C), as in teams that follow Behavior-Driven Development (BDD) or document requirements before implementation, may face different challenges. Our findings indicate that LLMs tend to produce tests more prone to structural errors and have a high incidence of test smells, especially *Suboptimal Assertions*. For teams working with agile methodologies and prioritizing natural language documentation, the use of LLMs can speed up test creation, but requires strategies to mitigate the low specificity of the assertions generated.

In addition, we observed differences in the performance of the LLMs in the different scenarios analyzed. GPT-4o, for example, had the lowest error rate and the greatest consistency in test generation, making it a viable option for teams seeking greater reliability in test automation. However, it had a high incidence of *Suboptimal Assertions*, which could impact teams using TDD or BDD, as it would

require adjustments to the verifications of each test generated. Amazon Q Developer, on the other hand, showed a high recurrence of structural errors, such as *IndentationError* and *NameError*, as well as several unique test smells. This suggests that its use may be more limited for scenarios that require well-structured tests. LLama had an intermediate performance, with a moderate number of errors, but stood out negatively for having the highest number of *AssertionError*. On the other hand, this model performed well when generating tests from text (T2C), which could be a plausible choice for developers working with documentation or BDD.

The adoption of LLMs for automatic test generation needs to be accompanied by good practices, such as manual review, more refined prompt engineering, and integration with static analysis tools. Companies using agile methodologies can benefit from these technologies to speed up test writing, but they need to consider the risks associated with the reliability of the tests generated, ensuring that automation does contribute to the continuous improvement of software quality.

7 Threats to Validity

- **Conclusion Validity** Although the tests were generated from the HumanEval dataset, the diversity of scenarios may not be wide enough to capture all the possible faults and test smells that LLMs can produce. Detecting errors and test smells alone may not be enough to accurately determine the quality of the tests generated. In addition, the analysis of these problems depends on automated tools and the language used, which have limitations and may not identify all the inconsistencies present in the tests.
- **Internal Validity** Variability in code generation by LLMs can influence the reproducibility of tests, especially if the models are updated in the future. In addition, possible differences in the parameters used by the LLMs may have impacted the quality of the tests generated, making it difficult to guarantee that the results are completely attributable to the types of prompts analyzed.
- **Construct Validity** Specific tools were used to detect test smells (Pynose and TEMPY), but they may not cover all smells in automated testing. In addition, the categorization of errors was based on messages from the Python interpreter, which can lead to different interpretations of the criticality of the problems encountered.
- **External Validity** The results may not be generalizable to other software domains, as the study was conducted exclusively with Python code and tests. In addition, only three LLMs were analyzed (GPT-4o, Amazon Q Developer and Llama 3.3), which may limit the applicability of the findings to other code generation models. The production codes present in HumanEval cover specific programming problems that are often used to train LLMs. As such, our results may not be generalizable to other code domains outside this scope, limiting the applicability of the findings to more complex or diverse contexts.

8 Conclusion and Future Work

In this paper, we conducted a comparative study of the quality of tests generated by three LLMs (GPT-4o, Amazon Q Developer and Llama 3.3). We used the HumanEval dataset to generate tests from prompts in the Text2Code and Code2Code contexts. After generation, the test suites were run alongside production code to check for errors. We then used test smells detection tools to assess the structural quality of the generated tests. In addition, we correlated the presence of errors and test smells in the same test suite.

Our results highlight the influence of the type of prompt and the LLM model on the quality of the test suites generated. The analysis revealed that the most frequent errors were *AssertionError*, *IndentationError* and *SyntaxError*, with significant variations between the models evaluated. GPT-4o had the lowest error rate, while Amazon Q Developer was the most likely to generate tests with flaws, including *IndentationError* and *NameError*, which were unique to this model. In addition, we identified that the most prevalent test smells were *Lack of Cohesion of Test Cases* and *Assertion Roulette*, indicating that many suites generated by LLMs suffer from low cohesion and multiple assertions without adequate contextualization. The

correlation between errors and test smells showed that structural problems have a direct impact on the reliability of automated tests.

Despite advances in automatic test generation, our findings suggest that LLMs still face challenges in producing high-quality suites, mainly due to the recurring presence of errors and violations of good testing practices. In view of this, in future studies we intend to explore different prompt engineering approaches, broaden the diversity of datasets evaluated and consider additional metrics for a more comprehensive analysis of the quality of tests generated by LLMs.

Artifact Availability

We provide our data and artifacts under open licenses at <https://github.com/ovictorpa/llms-research> and <https://zenodo.org/records/14773297>

Acknowledgments

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001; FAPESB grant PIE0002/2022; and CNPq grants 315840/2023-4 and 403361/2023-0.

References

- [1] Mayank Agarwal, Yikang Shen, Bailin Wang, Yoon Kim, and Jie Chen. 2024. Structured Code Representations Enable Data-Efficient Adaptation of Code Language Models. arXiv:2401.10716 [cs.CL] <https://arxiv.org/abs/2401.10716>
- [2] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, Brazil) (FSE 2024). Association for Computing Machinery, New York, NY, USA, 185–196. <https://doi.org/10.1145/3663529.3663839>
- [3] Victor Alves, Cristiano Santos, Carla Bezerra, and Ivan Machado. 2024. Detecting Test Smells in Python Test Code Generated by LLM: An Empirical Study with GitHub Copilot. In *Anais do XXXVIII Simpósio Brasileiro de Engenharia de Software* (Curitiba/PR). SBC, Porto Alegre, RS, Brasil, 581–587. <https://doi.org/10.5753/sbes.2024.3561>
- [4] Manoel Aranda, Naelson Oliveira, Elvys Soares, Márcio Ribeiro, Davi Romão, Ulysses Patriota, Rohit Gheyi, Emerson Souza, and Ivan Machado. 2024. A Catalog of Transformations to Remove Smells From Natural Language Tests. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* (Salerno, Italy) (EASE '24). Association for Computing Machinery, New York, NY, USA, 7–16. <https://doi.org/10.1145/3661167.3661225>
- [5] Alexandru Bodea. 2022. Pytest-Smell: a smell detection tool for Python unit tests. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual, South Korea) (ISSTA 2022). Association for Computing Machinery, New York, NY, USA, 793–796. <https://doi.org/10.1145/3533767.3543290>
- [6] W. Cain. 2024. In *Prompting Change: Exploring Prompt Engineering in Large Language Model AI and Its Potential to Transform Education*, Vol. 68. TechTrends, 47–57. <https://doi.org/10.1007/s11528-023-00896-0>
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgan Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. In *Evaluating Large Language Models Trained on Code*, Vol. abs/2107.03374. CoRR. arXiv:2107.03374 <https://arxiv.org/abs/2107.03374>
- [8] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In *Companion Proceedings of the 32nd ACM International Conference on the*

- Foundations of Software Engineering* (Porto de Galinhas, Brazil) (FSE 2024). Association for Computing Machinery, New York, NY, USA, 572–576. <https://doi.org/10.1145/3663529.3663801>
- [9] Tristan Coignon, Clément Quinton, and Romain Rouvoy. 2024. A Performance Study of LLM-Generated Code on LeetCode. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* (Salerno, Italy) (EASE '24). Association for Computing Machinery, New York, NY, USA, 79–89. <https://doi.org/10.1145/3661167.3661221>
 - [10] Ermira Daka and Gordon Fraser. 2014. In *A Survey on Unit Testing Practices and Problems*. 2014 IEEE 25th International Symposium on Software Reliability Engineering, 201–211. <https://doi.org/10.1109/ISSRE.2014.11>
 - [11] Humberto Damasceno, Carla Bezerra, Denivan Campos, Ivan Machado, and Emanuel Coutinho. 2023. Test smell refactoring revisited: What can internal quality attributes and developers' experience tell us? *Journal of Software Engineering Research and Development* 11, 1 (Oct. 2023), 13:1 – 13:16. <https://doi.org/10.5753/jsrerd.2023.3195>
 - [12] Khalid El Haji, Carolin Brandt, and Andy Zaidman. 2024. In *Using GitHub Copilot for Test Generation in Python: An Empirical Study* (Lisbon, Portugal) (AST '24). ACM, New York, NY, USA, 11. <https://doi.org/10.1145/3644032.3644443>
 - [13] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shu-vendu K. Lahiri. 2024. LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation. *IEEE Transactions on Software Engineering* 50, 9 (2024), 2254–2268. <https://doi.org/10.1109/TSE.2024.3428972>
 - [14] Daniel Fernandes, Ivan Machado, and Rita Maciel. 2022. TEMPY: Test Smell Detector for Python. In *Proceedings of the XXXVI Brazilian Symposium on Software Engineering* (Virtual Event, Brazil) (SBES '22). Association for Computing Machinery, New York, NY, USA, 214–219. <https://doi.org/10.1145/3555228.3555280>
 - [15] Danielle Gonzalez, Joanna C.S. Santos, Andrew Popovich, Mehdi Mirakhorli, and Mei Nagappan. 2017. A Large-Scale Study on the Usage of Testing Patterns That Address Maintainability Attributes: Patterns for Ease of Modification, Diagnoses, and Comprehension. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. 391–401. <https://doi.org/10.1109/MSR.2017.8>
 - [16] D. Graham, R. Black, and E. van Veenendaal. 2021. *Foundations of Software Testing ISTQB Certification, 4th edition*. Cengage Learning. <https://books.google.com.br/books?id=mOwxEAAAQBAJ>
 - [17] Emilia Hansson and Oliver Ellréus. 2023. *Code Correctness and Quality in the Era of AI Code Generation: Examining ChatGPT and GitHub Copilot*. Ph. D. Dissertation. <https://urn.kb.se/resolve?urn=urn:nbn:se:lnu:diva-121545>
 - [18] V. Khorikov. 2020. *Unit Testing Principles, Practices, and Patterns: Effective testing styles, patterns, and reliable automation for unit testing, mocking, and integration testing with examples in C#*. Manning. <https://books.google.com.br/books?id=CbvZyAEACAAJ>
 - [19] Dong Jae Kim. 2020. An Empirical Study on the Evolution of Test Smell. In *2020 IEEE/ACM 42nd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 149–151.
 - [20] Chun Li. 2022. In *Mobile GUI test script generation from natural language descriptions using pre-trained model* (Pittsburgh, Pennsylvania) (MOBILESoft '22). Association for Computing Machinery, New York, NY, USA, 112–113. <https://doi.org/10.1145/3524613.3527809>
 - [21] Chao Liu, Xuanlin Bao, Hongyu Zhang, Neng Zhang, Haibo Hu, Xiaohong Zhang, and Meng Yan. 2024. Guiding ChatGPT for Better Code Generation: An Empirical Study. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 102–113. <https://doi.org/10.1109/SANER60148.2024.00018>
 - [22] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. *arXiv:2102.04664 [cs.SE]* <https://arxiv.org/abs/2102.04664>
 - [23] P. Maragathavalli. 2011. Search-based software test data generation using evolutionary computation. *ArXiv abs/1103.0125* (2011). <https://api.semanticscholar.org/CorpusID:25209645>
 - [24] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende. 2024. Engenharia de Prompt em Grandes Modelos de Linguagem. In *Inteligência de Dados e Informática Cognitiva. ICDICI 2023 (Algoritmos para Sistemas Inteligentes)*, IJ Jacob, S. Piramuthu, and P. Falkowski-Gilski (Eds.). Springer, Cingapura. https://doi.org/10.1007/978-981-99-7962-2_30
 - [25] Noble Saji Mathews and Meiyappan Nagappan. 2024. Test-Driven Development and LLM-based Code Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1583–1594. <https://doi.org/10.1145/3691620.3695527>
 - [26] Phil McMinn. 2004. Search-based software test data generation: a survey: Research Articles. *Softw. Test. Verif. Reliab.* 14, 2 (jun 2004), 105–156.
 - [27] Jhonatan Oliveira, Luigi Mateus, Tássio Virgínio, and Larissa Rocha. 2024. SNUTS.js: Sniffing Nasty Unit Test Smells in Javascript. In *Anais do XXXVIII Simpósio Brasileiro de Engenharia de Software* (Curitiba/PR). SBC, Porto Alegre, RS, Brasil, 720–726. <https://doi.org/10.5753/sbes.2024.3563>
 - [28] Fabio Palomba, Andy Zaidman, and Andrea De Lucia. 2018. Automatic Test Smell Detection Using Information Retrieval Techniques. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 311–322. <https://doi.org/10.1109/ICSME.2018.00040>
 - [29] Zedong Peng, Xuanyi Lin, Michelle Simon, and Nan Niu. 2021. Unit and regression tests of scientific software: A study on SWMM. *Journal of Computational Science* 53 (2021), 101347. <https://doi.org/10.1016/j.jocs.2021.101347>
 - [30] Brent Reeves, Sami Sarsa, James Prather, Paul Denny, Brett A. Becker, Arto Hellas, Bailey Kimmel, Garrett Powell, and Juho Leinonen. 2023. Evaluating the Performance of Code Generation Models for Solving Parsons Problems With Small Prompt Variations. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1* (Turku, Finland) (ITICSE 2023). Association for Computing Machinery, New York, NY, USA, 299–305. <https://doi.org/10.1145/3587102.3588805>
 - [31] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM. *Proc. ACM Softw. Eng.* 1, FSE, Article 43 (July 2024), 21 pages. <https://doi.org/10.1145/3643769>
 - [32] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2024. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications. *arXiv:2402.07927 [cs.AI]* <https://arxiv.org/abs/2402.07927>
 - [33] Railana Santana, Luana Martins, Larissa Rocha, Tássio Virgínio, Adriana Cruz, Heitor Costa, and Ivan Machado. 2020. RAIDE: a tool for Assertion Roulette and Duplicate Assert identification and refactoring. In *Proceedings of the XXXIV Brazilian Symposium on Software Engineering*. 374–379.
 - [34] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. In *An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation*, Vol. 50. IEEE Transactions on Software Engineering, 85–105. <https://doi.org/10.1109/TSE.2023.3334955>
 - [35] Sina Shamshiri, José Miguel Rojas, Juan Pablo Galeotti, Neil Walkinshaw, and Gordon Fraser. 2018. How Do Automatically Generated Unit Tests Influence Software Maintenance?. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*. 250–261. <https://doi.org/10.1109/ICST.2018.00033>
 - [36] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2024. Using Large Language Models to Generate JUnit Tests: An Empirical Study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* (Salerno, Italy) (EASE '24). Association for Computing Machinery, New York, NY, USA, 313–322. <https://doi.org/10.1145/3661167.3661216>
 - [37] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2021. Unit Test Case Generation with Transformers and Focal Context. *arXiv:2009.05617 [cs.SE]*
 - [38] Michele Tufano, Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, Andrea De Lucia, and Denys Poshyvanyk. 2016. An empirical investigation into the nature of test smells. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering* (Singapore, Singapore) (ASE '16). Association for Computing Machinery, New York, NY, USA, 4–15. <https://doi.org/10.1145/2970276.2970340>
 - [39] Arie van Deursen, Leon Moonen, Alex van den Bergh, and Gerard Kok. 2001. In *Refactoring Test Code*, M. Marchesi and G. Succi (Eds.). Proceedings 2nd International Conference on Extreme Programming and Flexible Processes in Software Engineering (XP2001).
 - [40] Tongjie Wang, Yaroslav Golubev, Oleg Smirnov, Jiawei Li, Timofey Bryksin, and Iftekhar Ahmed. 2022. In *PyNose: a test smell detector for python* (Melbourne, Australia) (ASE '21). IEEE Press, 593–605. <https://doi.org/10.1109/ASE51524.2021.9678615>
 - [41] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 1258–1268. <https://doi.org/10.1145/3691620.3695501>
 - [42] Tao Xie and David Notkin. 2006. Tool-assisted unit test generation and selection based on operational abstractions. *Automated Software Engineering Journal* 13, 3 (July 2006), 345–371.
 - [43] Burak Yetistiren, Isik Ozsoy, and Eray Tuzun. 2022. Assessing the quality of GitHub copilot's code generation. In *Assessing the quality of GitHub copilot's code generation* (Singapore, Singapore) (PROMISE 2022). Association for Computing Machinery, New York, NY, USA, 62–71. <https://doi.org/10.1145/3558489.3559072>
 - [44] Burak Yetistiren, Isik Ozsoy, Miray Ayerdem, and Eray Tüzün. 2023. In *Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT*. <https://doi.org/10.48550/arXiv.2304.10778>
 - [45] Shengcheng Yu, Chunrong Fang, Yucheng Ling, Chentian Wu, and Zhenyu Chen. 2023. LLM for Test Script Generation and Migration: Challenges, Capabilities, and Opportunities. In *LLM for Test Script Generation and Migration*:

Challenges, Capabilities, and Opportunities. 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS), 206–217. <https://api.semanticscholar.org/CorpusID:262459296>

[//api.semanticscholar.org/CorpusID:262459296](https://api.semanticscholar.org/CorpusID:262459296)